UNIVERSITI TEKNOLOGI MALAYSIA

**FACULTY OF COMPUTING**

**SECJ2013**

**DATA STRUCTURES AND ALGORITHMS**

**SECTION 02, 2024/2025**

# MINI PROJECT:

# Menu Driven Stack Application using Browser Tabs

**LECTURER'S NAME: DR ZURAINI ALI SHAH**

**GROUP 4**

| BIL | GROUP MEMBERS | MATRIC NO . |
|---|---|---|
| 1 | LUQMAN HARITH BIN MOHD SHAHAR | A23CS3009 |
| 2 | LUVINESH SUDESH | A21EC0198 |
| 3 | GOE JIE YING | A23CS0224 |
| 4 | BASEL A. A. ABUNADA | A22EC4006 |

YOUTUBE VIDEO LINK DEMO: https://youtu.be/e8fA_QSHpSI

**1.0 Introduction** **3**
1.1 Synopsis 3
1.2 Objective 4
**2.0 System Analysis and Design** **5**
2.1 System Requirement 5
2.1.1 Functional Requirement 5
2.1.2 Non-Functional Requirements 5
2.1.3 Use Case Diagram 6
2.2 System Design 7
2.2.1 Design Class Diagram 7
Stack Class Design: 7
Attributes (Data Members): 7
Methods (Member Functions): 7
2.2.2 Class Diagram 8
**3.0 Development Activities** **9**
3.1 Few Examples of development activities 9
3.2 Output of C++ Program: 12
**4.0 Task Division** **14**
4.1 Individual Contribution 14
4.2 Meeting Log 15

# 1.0 Introduction

## 1.1 Synopsis

The Browser Tab system is created to provide an user-friendly interface to manage the tabs. It allows the user to open a new tab for searching purposes, close a tab after searching and restore the closed tab.

Here are some key features in this system:

1. Add Tab
   Users can open a new tab whenever they want to search for something in the browser. The user can open up to 10 tabs.

2. Close a Tab
   - Users can close a tab once they have finished their search.
   - The tabs should be closed individually.
   - When the user closes all the tabs, the system will automatically close.

3. Recover a Tab
   Users can recover the most recently closed tab, followed by the next one, based on the Last In, First Out (LIFO) principle.

## 1.2 Objective

The primary objective of choosing "Stack Application - menu driven" as our topic is to design a simple application that simulates the functionalities of the stack data structure. It was chosen due to its versatility and simplicity of showcasing stack data structure (Last-In-First-Out).

The system is expected to achieve the following goals:

1. Demonstrate Stack Operation
    - Apply the Last In First Out (LIFO) principle
    - Include fundamental Stack functions such as createStack(), push(), pop(), isEmpty(), isFull(), stackTop()

2. Simplify User Interaction
    - Provide a menu-driven interface
    - Ensure the system is user friendly
    - Even first-time user can operate it effortlessly

# 2.0 System Analysis and Design

## 2.1 System Requirement

### 2.1.1 Functional Requirement

1. Open New Tab
    - The system must allow the user to open a new tab by entering the tab name
    - The system should prevent opening a new tab if reach the max limit (10 tabs)

2. Close Tab
    - Users must be able to close the most recently opened tab
    - Closed tabs should be saved to an undo stack for recovery
    - If the user deletes all of the available tabs, the system should close by itself
3. Recover Closed Tab
    - Must allows user to restore the most recently closed tab
    - Follow the Last In First Out (LIFO) principle

4. Display Open Tabs
    - The system should display the tabs after entering the name of website

5. Display Top of the Stack
    - The system must be able to show the top tab (stackTop()) upon user requirement

6. Exit Browser
    - After browsing, the user can choose to exit the browser. If there are multiple browser tabs opened, the system will double ask the user whether they like to exit or not.

### 2.1.2 Non-Functional Requirements

1. Usability
    - The menu and prompts should be user friendly and intuitive
    - Error messages should be clear and guide the user to correct the error.
2. Reliability
    - The system must ensure data integrity
    - The undo stack should not overflow beyond its capacity

3. Maintainability
    - The code should be modular and easy to modify

4. Portability
    - The system should work on any platform that supports a C++ compiler.

### 2.1.3 Use Case Diagram

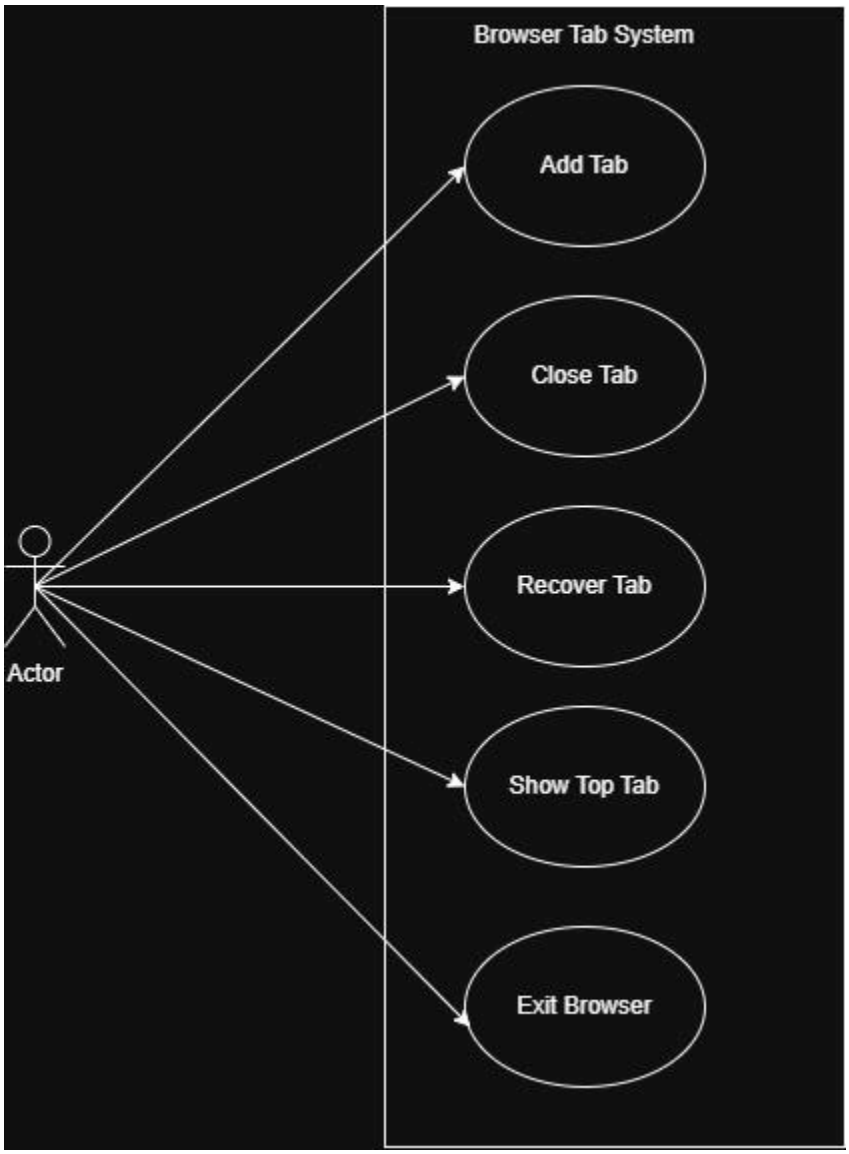


Figure: Use Case Diagram of Browser Tab System

The user will be able to perform 5 activities in the system

## 2.2 System Design

### 2.2.1 Design Class Diagram

**Stack Class Design:**

- Purpose: Manages browser tabs using a LIFO (Last In First Out) data structure
- Memory Management:
  - Uses fixed-size arrays for predictable memory usage
  - Separate arrays for active and closed tabs
- Access Control:
  - Private data members for encapsulation
  - Public interface for tab operations

**Attributes (Data Members):**

- Main Stack Components:
  - top: Integer index tracking current active tab position (-1 to 9)
  - data: Fixed array storing active tab names (max 10 tabs)
- Undo Stack Components:
  - undoTop: Integer index for recently closed tabs (-1 to 9)
  - undoData: Fixed array storing closed tab names (max 10 tabs)

**Methods (Member Functions):**

- Stack Operations:
  - createStack(): Initializes both stacks with -1 as top indices
  - push(): Adds new tab if space available
  - pop(): Removes current tab and stores in undo stack
  - stackTop(): Returns the top element of a stack
- Tab Management:
  - recoverTab(): Restores most recently closed tab
  - displayStack(): Shows all open tabs in horizontal format

- State Validation:
  - isFull(): Prevents stack overflow
  - isEmpty(): Checks for empty stack
  - isUndoFull(): Manages undo stack capacity
  - isUndoEmpty(): Validates undo operations

- Utility Functions:
  - getTabCount(): Returns current number of open tabs

### 2.2.2 Class Diagram

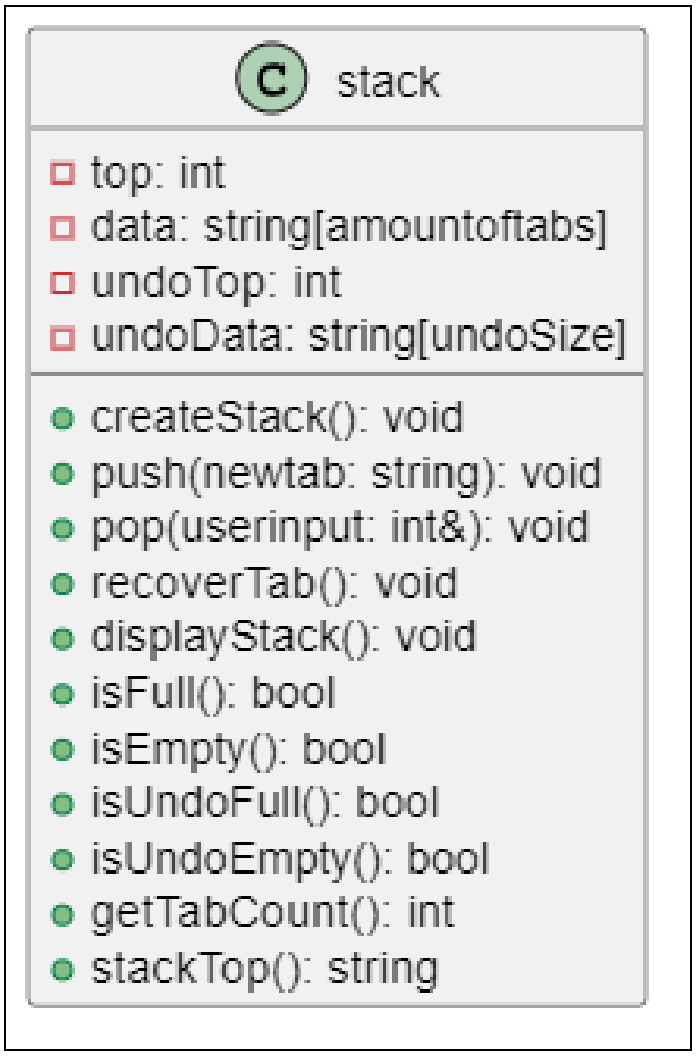


Figure: Class Diagram for Browser Tab System

Since there is only **one** class in this system, there is no relationship between classes. The stack class will have 4 private attributes and 11 public methods.

# 3.0 Development Activities

## 3.1 Few Examples of development activities

- Developing the diagrams

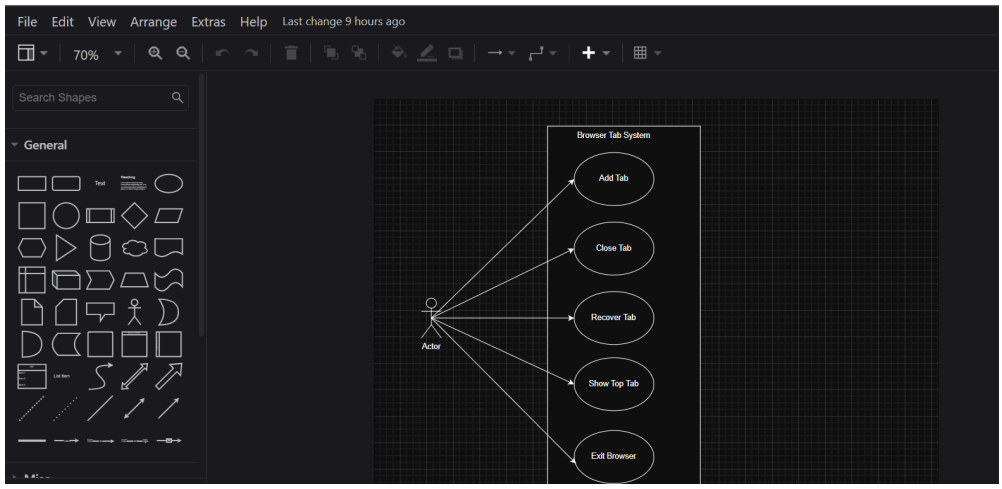


Figure: Using draw.io to create the use case diagrams

- Display Function

```
void displayStack()
{
    if (isEmpty())
    {
        cout << "No open tabs." << endl;
    }
    else
    {
        cout << "\n-- OPENED TABS --" << endl;
        for (int i = 0; i <= top; i++) // Display tabs from bottom to top horizontally
        {
            cout << "| " << i + 1 << ". " << data[i] << " ";
        }
        cout << "|" << endl; // End with a pipe for formatting
    }
}
```

Figure: Overall displayStack Logic

To simulate the appearance of a web browser, the displayStack function will print out the stacks in a horizontal manner. This is achieved by implementing a for loop.

- Pop Function

```
void pop(int &userinput)
{
    if (isEmpty())
    {
        cout << "Cannot close a tab, stack is EMPTY." << endl;
    }
    else
    {
        cout << "Closed tab: " << data[top] << endl;

        // Save the closed tab into the undo stack
        if (!isUndoFull())
        {
            undoTop += 1;
            undoData[undoTop] = data[top];
        }
        else
        {
            cout << "Undo stack is full. Cannot save this closed tab." << endl;
        }

        top -= 1; // Remove the tab from the main stack

        // Check if the stack is empty and trigger exit
        if (isEmpty())
        {
            cout << "All tabs are closed. Exiting browser automatically..." << endl;
            userinput = 5; // Trigger exit condition
        }
    }
}
```

Figure: Overall Pop Function Logic

The pop() function is arguably the most complex feature of this system. To add complexity and realism to our mini project, the `pop` function utilizes a pass-by-reference parameter to manage system behavior dynamically. This approach ensures that the system can automatically shut down when all available browser tabs are closed. By passing the user input by reference, the function can directly modify the value of `userinput` within the calling scope. For instance, if there are three browser tabs open and the user closes all of them, the `pop` function updates `userinput` to trigger the system's exit condition. This seamless integration between the function's logic and the main program flow highlights the necessity of using pass-by-reference in scenarios where real-time updates to external variables are essential.

- Usage of switch statements instead of if else statements

```
case 1: // Add a tab
    cout << "Enter tab name: ";
    cin.ignore(); // Clear input buffer
    getline(cin, tabName);
    browserStack.push(tabName);
    break;

case 2: // Close a tab
    browserStack.pop(userinput);
    break;

case 3: // Recover a tab
    browserStack.recoverTab();
    break;

case 4: // Display the top tab
    if (!browserStack.isEmpty())
    {
        cout << "The top tab is: " << browserStack.stackTop() << endl;
    }
    break;

case 5:                                 // Exit with logic
    if (browserStack.getTabCount() == 1) // If only one tab is open
    {
        cout << "One tab open. Closing browser automatically..." << endl;
    }
    else if (browserStack.getTabCount() > 1) // If more than one tab is open
    {
        char confirm;
        cout << "You have multiple open tabs. Are you sure you want to close the browser? (y/n): ";
        cin >> confirm;

        if (confirm == 'y' || confirm == 'Y')
        {
            cout << "Exiting browser..." << endl;
        }
        else
        {
            userinput = 0; // Reset input to stay in the loop
            cout << "Exit cancelled. Returning to menu..." << endl;
        }
```

Figure: 5 switch  statements

Switch statements are preferred over repetitive if-else statements because they improve code clarity, efficiency and maintainability.

## 3.2 Output of C++ Program:

```
-- -- -- OPERA BROWSER -- -- -- -- --
No open tabs.

Menu Options:
1. Add Tab
2. Close Tab
3. Recover Tab
4. Show Top Tab
5. Exit Browser
Enter your choice:
```

Figure: System Menu

```
-- -- -- OPERA BROWSER -- -- -- -- --

-- OPENED TABS --
| 1. UTM | 2. UKM | 3. USM |

Menu Options:
1. Add Tab
2. Close Tab
3. Recover Tab
4. Show Top Tab
5. Exit Browser
Enter your choice:
```

Figure: Option 1 (Add Tab/Push)

```
-- -- -- OPERA BROWSER -- -- -- -- --

-- OPENED TABS --
| 1. UTM |

Menu Options:
1. Add Tab
2. Close Tab
3. Recover Tab
4. Show Top Tab
5. Exit Browser
Enter your choice:
```

Figure: Option 2 (Close Tab/Pop)

```
-- -- -- OPERA BROWSER -- -- -- -- --

-- OPENED TABS --
| 1. UTM | 2. UKM | 3. USM |

Menu Options:
1. Add Tab
2. Close Tab
3. Recover Tab
4. Show Top Tab
5. Exit Browser
Enter your choice:
```

Figure: Option 3 (Recover Closed Tabs)

```
Enter your choice: 4
The top tab is: USM
```

Figure: Option 4 (Show Top Tab/StackTop)

```
-- -- -- OPERA BROWSER -- -- -- -- --

-- OPENED TABS --
| 1. UTM |

Menu Options:
1. Add Tab
2. Close Tab
3. Recover Tab
4. Show Top Tab
5. Exit Browser
Enter your choice: 5
One tab open. Closing browser automatically...
```

Figure: Option 5 (Exiting Browser)

# 4.0 Task Division

## 4.1 Individual Contribution

| Name | Task |
|---|---|
| Luqman | ● Develop the codes<br>● Record the input and output of the codes |
| Luvinesh | ● Code checking<br>● Use case diagram |
| Goe Jie Ying | ● Code checking<br>● Synopsis<br>● Objective<br>● System Requirement |
| Basel | ● Code checking<br>● Design Class Diagram<br>● Class Diagram |

| Date | Task Name | Contributor Name | Hours Worked | Comment |
|---|---|---|---|---|
| 30/12/2024 | Decide our first topic | Luvi, Luqman, Goe | 0.5 | First topic decided: LIFO concept and implementation in visiting the web (a simulation). |
| 31/12/2024 | Research On Topic | Luvi, Luqman, Goe, Basel | 3 | Decided to change the topic |
| 10/1/2025 | Discuss with Dr. | Luqman, Goe | 0.3 | Dr, agreed on the topic changing |
| 11/1/2025 | Trying on code | Luqman, Goe | 2.5 | Try to understand the concept and working on codes, Luqman's idea was chosen |

| | | | | |
|---|---|---|---|---|
| 12/1/2025 | Task Division | Luvi, Luqman, Goe, Basel | 0.5 | Goe - Synopsis, Objectives and system requirement<br>Luvi - Use Cases Diagram<br>Basel - System Design and Class Diagram<br>Luqman - Coding (Input & Output) |
| 12/1/2025 | Class Diagram | Basel | 2.5 | Successfully designed the class diagram and finished the class diagram |
| 13/1/2024 | Synopsis and Objective | Goe | 1 | Outlined the synopsis and stated the objectives |
| 13/1/2024 | Use Case Diagram | Luvi | 2 | Completed the use case diagram for our system |
| 15/1/2024 | System Requirement | Goe | 1 | Finished outline the system requirement for our system |
| 17/1/2024 | Code Implementation | Luqman | 3 | Finalised the codes and record it's input and output in the documents. |

## 4.2 Meeting Log

| Date | Member Participated | Agenda | Action Items |
|---|---|---|---|
| 30/12/2024 | Luqman<br>Luvinesh<br>Goe<br>Basel (Absent - Hospital Appointment) | 1. Review the mini project | 1. Create the project outline<br>2. Discuss about the title next meeting<br>3. Task Distribution |
| 12/1/2025 | Luqman<br>Luvinesh<br>Goe<br>Basel | 1. Discuss about Luqman's idea<br>2. Understand the idea<br>3. Task Division | Goe - Synopsis, Objectives and system requirement<br>Luvi - Use Cases Diagram<br>Basel - System Design and Class Diagram<br>Luqman - Coding (Input & Output) |